

claude-windows / 42211f80-2169-4025-ad9d-99c1aa29ff70

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\42211f80-2169-4025-ad9d-99c1aa29ff70\subagents\workflows\wf_c03ffa6c-cb8\agent-a09792ac9847f73a1.jsonl`
- SHA-256: `f7473f28bfc2726eda5f342ffa6ea9c1e6ba6ad395b5234f3bd7645e005a48e`
- Source modified: `2026-06-06T17:33:29+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf_c03ffa6c-cb8`
- session_id: `42211f80-2169-4025-ad9d-99c1aa29ff70`
```

Transcript

1. user (2026-06-06T17:32:34.152Z)

Extract the EXACT annotation schema the VLC tool must emit so collector 'import-local' ingests it with zero transformation. Read: 'C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py' (the `_parse_local_annotations` method, CSV branch ? exact lowercased column names it reads), and the docs 'C:/Users/avidu/Projects/sports-data-collector/docs/annotation/ANNOTATION_GUIDE.md' and 'R00RA_BRIEF.md' for the ending_reason vocabulary and any last_shot_outcome / shots_count fields. Report: the precise CSV header columns + units (seconds), the allowed ending_reason values, optional fields, and a one-line example row. Cite file:line.

2. assistant (2026-06-06T17:32:36.991Z)

I'll read the three files to extract the exact annotation schema.

3. user (2026-06-06T17:32:38.563Z)

```
1      import os
2      import sys
3      import time
4      import json
5      import csv
6      import argparse
7      from typing import List, Dict, Any, Tuple
8      from tabulate import tabulate
9
10     from src.core.config import load_config
11     from src.core.parsing import safe_int as _safe_int, safe_float_required as _safe_float
12     from src.core.db import SportsDatabase
13     from src.core.schemas.base import VideoMetadata, SportType, LicenseType, CurationStatus
14     from src.core.schemas.badminton import BadmintonRally
15     from src.core.schemas.tennis import TennisPoint
16     from src.core.schemas.cricket import CricketDelivery
17     from src.core.schemas.pickleball import PickleballRally
18     from src.core.schemas.tabletennis import TableTennisRally
19     from src.core.validator import DatasetValidator
20     from src.core.downloader import download_best_video, probe_resolution_fps,
height_to_label
21
22
23     class SportsDataCLI:
24         def __init__(self, config_path: str = "config/config.yaml"):
25             self.config_path = config_path
26             self.config = self._load_config()
27
28             # Resolve DB path
```

```

29         db_path = self.config.get("database_path", "data/sports_metadata.db")
30         self.db = SportsDatabase(db_path)
31         self.validator = DatasetValidator(config_path)
32
33     def _load_config(self) -> Dict[str, Any]:
34         if not os.path.exists(self.config_path):
35             print(f"Warning: Configuration file not found at {self.config_path}. Using
defaults.")
36         return load_config(self.config_path)
37
38     def status(self):
39         """Displays status summary of all videos in the database."""
40         # Query total count by status via the public DB read API.
41         rows = self.db.get_status_counts() # list of (sport, status, cnt)
42
43         if not rows:
44             print("\nDatabase is currently empty. Run crawlers or import local data.\n")
45             return
46
47         data = [[sport, status, cnt] for sport, status, cnt in rows]
48         print("\n=== Dataset Ingestion Status ===")
49         print(tabulate(data, headers=["Sport", "Status", "Count"], tablefmt="grid"))
50         print()
51
52     def import_local(self, args):
53         """Manually registers a local video and annotation file into the database."""
54         sport_str = args.sport.lower()
55         if sport_str not in self.config.get("supported_sports", ["badminton"]):
56             print(f"Error: Sport '{sport_str}' is not supported. Supported:
{self.config.get('supported_sports')}")
57             sys.exit(1)
58
59         # Validate video path
60         if not os.path.exists(args.video_path):
61             print(f"Warning: Local video path '{args.video_path}' does not exist on
disk, but continuing registration.")
62
63         # Validate annotation file
64         if not os.path.exists(args.annotations_path):
65             print(f"Error: Annotations file '{args.annotations_path}' not found.")
66             sys.exit(1)
67
68         match_name = args.match_name or os.path.basename(args.video_path).split('.')[0]
69         video_id = args.video_id or match_name.lower().replace(" ", "_")
70
71         # Parse license
72         try:
73             license_type = LicenseType(args.license)
74         except ValueError:
75             print(f"Warning: Unknown license '{args.license}'. Defaulting to Unknown
(non-commercial).")
76             license_type = LicenseType.UNKNOWN
77
78         is_commercial = self.validator.check_license_commercial(license_type)
79
80         # 1. Parse Annotations
81         annotations = []
82         try:
83             annotations = self._parse_local_annotations(sport_str, video_id,
args.annotations_path)
84         except Exception as e:
85             print(f"Error parsing annotations: {e}")
86             sys.exit(1)
87
88         # 2. Run Validation

```

```

89         is_valid = True
90         errors = []
91         if sport_str == "badminton":
92             is_valid, errors = self.validator.validate_badminton_rallies(annotations)
93         elif sport_str == "tennis":
94             is_valid, errors = self.validator.validate_tennis_points(annotations)
95         elif sport_str == "cricket":
96             is_valid, errors = self.validator.validate_cricket_deliveries(annotations)
97         elif sport_str == "pickleball":
98             is_valid, errors = self.validator.validate_pickleball_rallies(annotations)
99         elif sport_str == "tabletennis":
100             is_valid, errors = self.validator.validate_tabletennis_rallies(annotations)
101
102     if not is_valid:
103         print(f"Error: Annotations failed validation!")
104         for err in errors:
105             print(f" - {err}")
106         sys.exit(1)
107
108     # 3. Create Video Metadata
109     video_meta = VideoMetadata(
110         video_id=video_id,
111         sport=SportType(sport_str),
112         video_url=None,
113         local_path=args.video_path,
114         license_type=license_type,
115         license_url=None,
116         is_commercial=is_commercial,
117         status=CurationStatus.CURATED, # Custom local imports bypass staging by
default
118         match_name=match_name,
119         fps=args.fps,
120         resolution=args.resolution
121     )
122
123     # 4. Insert into Database
124     self.db.insert_video(video_meta)
125
126     count = 0
127     if sport_str == "badminton":
128         count = self.db.insert_badminton_rallies(annotations)
129     elif sport_str == "tennis":
130         count = self.db.insert_tennis_points(annotations)
131     elif sport_str == "cricket":
132         count = self.db.insert_cricket_deliveries(annotations)
133     elif sport_str == "pickleball":
134         count = self.db.insert_pickleball_rallies(annotations)
135     elif sport_str == "tabletennis":
136         count = self.db.insert_tabletennis_rallies(annotations)
137
138     print(f"\nSuccessfully imported local video '{match_name}':")
139     print(f" - Video ID: {video_id}")
140     print(f" - Sport: {sport_str}")
141     print(f" - Commercial Use Allowed: {is_commercial} (License:
{license_type.value})")
142     print(f" - Registered {count} annotated segments.\n")
143
144     def _parse_local_annotations(self, sport: str, video_id: str, filepath: str) ->
List[Any]:
145         """Parses simple JSON or CSV annotations files."""
146         parsed_items = []
147         ext = os.path.splitext(filepath)[1].lower()
148
149         if ext == '.json':
150             with open(filepath, 'r', encoding='utf-8') as f:

```

```

151         data = json.load(f)
152
153     if not isinstance(data, list):
154         raise ValueError("JSON file must contain a list of objects.")
155
156     # Uses the same safe casts as the CSV path: a missing/blank required
157     # time raises a clear ValueError (caught by the caller) instead of a
158     # bare KeyError, and optional ints degrade to sensible defaults.
159     for idx, obj in enumerate(data):
160         if sport == "badminton":
161             parsed_items.append(BadmintonRally(
162                 video_id=video_id,
163                 rally_number=_safe_int(obj.get("rally_number"), idx + 1),
164                 start_time=_safe_float(obj.get("start_time")),
165                 end_time=_safe_float(obj.get("end_time")),
166                 score_before=obj.get("score_before"),
167                 score_after=obj.get("score_after"),
168                 shots_count=_safe_int(obj.get("shots_count"), None),
169                 ending_reason=obj.get("ending_reason"),
170                 last_shot_outcome=obj.get("last_shot_outcome")
171             ))
172         elif sport == "tennis":
173             parsed_items.append(TennisPoint(
174                 video_id=video_id,
175                 point_number=_safe_int(obj.get("point_number"), idx + 1),
176                 start_time=_safe_float(obj.get("start_time")),
177                 end_time=_safe_float(obj.get("end_time")),
178                 set_score=obj.get("set_score"),
179                 game_score=obj.get("game_score"),
180                 server=obj.get("server"),
181                 ending_type=obj.get("ending_type")
182             ))
183         elif sport == "cricket":
184             parsed_items.append(CricketDelivery(
185                 video_id=video_id,
186                 over=_safe_int(obj.get("over"), 0),
187                 ball=_safe_int(obj.get("ball"), 0),
188                 start_time=_safe_float(obj.get("start_time")),
189                 end_time=_safe_float(obj.get("end_time")),
190                 batsman=obj.get("batsman"),
191                 bowler=obj.get("bowler"),
192                 runs=_safe_int(obj.get("runs"), None),
193                 wicket=bool(obj.get("wicket", False)),
194                 extras=_safe_int(obj.get("extras"), 0)
195             ))
196         elif sport == "pickleball":
197             parsed_items.append(PickleballRally(
198                 video_id=video_id,
199                 rally_number=_safe_int(obj.get("rally_number"), idx + 1),
200                 start_time=_safe_float(obj.get("start_time")),
201                 end_time=_safe_float(obj.get("end_time")),
202                 score_before=obj.get("score_before"),
203                 score_after=obj.get("score_after"),
204                 shots_count=_safe_int(obj.get("shots_count"), None),
205                 ending_reason=obj.get("ending_reason"),
206                 last_shot_outcome=obj.get("last_shot_outcome")
207             ))
208         elif sport == "tabletennis":
209             parsed_items.append(TableTennisRally(
210                 video_id=video_id,
211                 rally_number=_safe_int(obj.get("rally_number"), idx + 1),
212                 start_time=_safe_float(obj.get("start_time")),
213                 end_time=_safe_float(obj.get("end_time")),
214                 score_before=obj.get("score_before"),
215                 score_after=obj.get("score_after"),

```

```

216             shots_count=_safe_int(obj.get("shots_count"), None),
217             ending_reason=obj.get("ending_reason"),
218             last_shot_outcome=obj.get("last_shot_outcome")
219         ))
220     elif ext == '.csv':
221         with open(filepath, 'r', encoding='utf-8') as f:
222             reader = csv.DictReader(f)
223             # Normalize headers
224             reader.fieldnames = [n.lower().strip() for n in reader.fieldnames] if
reader.fieldnames else []
225
226         for idx, row in enumerate(reader):
227             if sport == "badminton":
228                 parsed_items.append(BadmintonRally(
229                     video_id=video_id,
230                     rally_number=_safe_int(row.get("rally_number"), idx + 1),
231                     start_time=_safe_float(row.get("start_time")),
232                     end_time=_safe_float(row.get("end_time")),
233                     score_before=row.get("score_before"),
234                     score_after=row.get("score_after"),
235                     shots_count=_safe_int(row.get("shots_count"), None),
236                     ending_reason=row.get("ending_reason"),
237                     last_shot_outcome=row.get("last_shot_outcome")
238                 ))
239             elif sport == "tennis":
240                 parsed_items.append(TennisPoint(
241                     video_id=video_id,
242                     point_number=_safe_int(row.get("point_number"), idx + 1),
243                     start_time=_safe_float(row.get("start_time")),
244                     end_time=_safe_float(row.get("end_time")),
245                     set_score=row.get("set_score"),
246                     game_score=row.get("game_score"),
247                     server=row.get("server"),
248                     ending_type=row.get("ending_type")
249                 ))
250             elif sport == "cricket":
251                 parsed_items.append(CricketDelivery(
252                     video_id=video_id,
253                     over=_safe_int(row.get("over"), 0),
254                     ball=_safe_int(row.get("ball"), 0),
255                     start_time=_safe_float(row.get("start_time")),
256                     end_time=_safe_float(row.get("end_time")),
257                     batsman=row.get("batsman"),
258                     bowler=row.get("bowler"),
259                     runs=_safe_int(row.get("runs"), None),
260                     wicket=(row.get("wicket") or "").lower() in ["true", "1",
"yes"],
261                     extras=_safe_int(row.get("extras"), 0)
262                 ))
263             elif sport == "pickleball":
264                 parsed_items.append(PickleballRally(
265                     video_id=video_id,
266                     rally_number=_safe_int(row.get("rally_number"), idx + 1),
267                     start_time=_safe_float(row.get("start_time")),
268                     end_time=_safe_float(row.get("end_time")),
269                     score_before=row.get("score_before"),
270                     score_after=row.get("score_after"),
271                     shots_count=_safe_int(row.get("shots_count"), None),
272                     ending_reason=row.get("ending_reason"),
273                     last_shot_outcome=row.get("last_shot_outcome")
274                 ))
275             elif sport == "tabletennis":
276                 parsed_items.append(TableTennisRally(
277                     video_id=video_id,
278                     rally_number=_safe_int(row.get("rally_number"), idx + 1),

```

```

279         start_time=_safe_float(row.get("start_time")),
280         end_time=_safe_float(row.get("end_time")),
281         score_before=row.get("score_before"),
282         score_after=row.get("score_after"),
283         shots_count=_safe_int(row.get("shots_count"), None),
284         ending_reason=row.get("ending_reason"),
285         last_shot_outcome=row.get("last_shot_outcome")
286     ))
287     else:
288         raise ValueError("Unsupported annotations file extension. Use .json or
.csv.")
289
290     return parsed_items
291
292 def _fetch_one_with_retry(self, url, dest_path, args, max_height, max_size_mb):
293     """Download one video at best resolution, retrying transient failures.
294
295     A below-min-height result almost always means YouTube rate-limited the
296     request and served a low fallback (not that the video lacks HD), so we
297     retry with a growing backoff. Returns (DownloadResult, None) on success,
298     or (None, failure_reason) once retries are exhausted. A below-min file is
299     discarded each attempt so it never lands in the DB.
300     """
301     attempts = max(1, args.retries + 1)
302     reason = "unknown error"
303     for i in range(attempts):
304         result = download_best_video(
305             url, dest_path,
306             max_height=max_height, max_size_mb=max_size_mb,
307             cookies_from_browser=args.cookies_from_browser,
308             cookies_file=args.cookies,
309             player_client=args.player_client,
310         )
311         if result.success and (result.height is None or result.height >=
args.min_height):
312             return result, None
313
314         if not result.success:
315             reason = result.error or "unknown error"
316         else:
317             reason = (f"got {result.resolution_label} (< min-height
{args.min_height})p) ? "
318                 f"YouTube served a low format (rate-limited?)")
319         try:
320             os.remove(result.path)
321         except OSError:
322             pass
323
324         if i < attempts - 1:
325             backoff = args.retry_backoff * (i + 1)
326             print(f" attempt {i+1}/{attempts} failed: {reason}\n retrying in
{backoff:.0f}s...")
327             time.sleep(backoff)
328     return None, reason
329
330 def fetch(self, args):
331     """(Re)download videos at best available resolution, updating the DB in place.
332
333     This is the in-place upgrade path: rows, rally annotations, and curation
334     status are all preserved ? only the video file plus its `local_path`,
335     `resolution`, and `fps` are refreshed. By default it skips videos that
336     already have a local file at or above `--min-height`, so re-runs are cheap
337     and resumable; `--force` re-fetches everything.
338     """
339     storage = self.config.get("video_storage", {})

```

```

340         local_dir = storage.get("local_dir", "./data/videos")
341         max_download_size_mb = storage.get("max_download_size_mb")
342         max_height = args.max_height if args.max_height is not None else
storage.get("max_height")
343         os.makedirs(local_dir, exist_ok=True)
344
345         commercial_only = not args.include_noncommercial
346         if args.status == "all":
347             statuses = [CurationStatus.CURATED, CurationStatus.STAGING]
348         else:
349             statuses = [CurationStatus(args.status)]
350
351         supported = self.config.get("supported_sports", ["badminton"])
352         sport_strs = [args.sport] if args.sport else supported
353
354         # Collect target videos.
355         videos = []
356         for sport_str in sport_strs:
357             try:
358                 sport = SportType(sport_str.lower())
359             except ValueError:
360                 print(f"Warning: skipping unknown sport '{sport_str}'.")
361                 continue
362             for status in statuses:
363                 videos.extend(self.db.get_videos_by_status(status, sport))
364         if args.video_id:
365             videos = [v for v in videos if v.video_id == args.video_id]
366
367         if not videos:
368             print("\nNo matching videos to fetch.\n")
369             return
370
371         results = [] # (video_id, action, detail)
372         for video in videos:
373             if commercial_only and not video.is_commercial:
374                 results.append((video.video_id, "skip", "non-commercial"))
375                 continue
376             if not video.video_url or "watch?v=example" in video.video_url:
377                 # Either no URL, or the parser's placeholder (the ShuttleSet catalog
378                 # had no URL for this match) ? nothing to download.
379                 results.append((video.video_id, "skip", "no real source URL"))
380                 continue
381
382             dest_path = os.path.join(local_dir, f"{video.video_id}.mp4")
383
384             # Skip if we already have a good-enough local file (unless --force).
385             if not args.force and os.path.exists(dest_path):
386                 _, height, _ = probe_resolution_fps(dest_path)
387                 if height is not None and height >= args.min_height:
388                     results.append((video.video_id, "have", f"{height_to_label(height)}
already"))
389                     continue
390
391             print(f"Fetching {video.video_id} ({video.match_name})...")
392             result, fail_reason = self._fetch_one_with_retry(
393                 video.video_url, dest_path, args, max_height, max_download_size_mb)
394             if fail_reason is not None:
395                 results.append((video.video_id, "FAIL", fail_reason))
396                 if args.sleep:
397                     time.sleep(args.sleep)
398                 continue
399
400             # Update only the file-derived fields; preserve everything else (UPSERT
401             # mutates the row in place, leaving rallies intact).
402             video.local_path = result.path

```

```

403         if result.resolution_label:
404             video.resolution = result.resolution_label
405         if result.fps:
406             video.fps = result.fps
407         self.db.insert_video(video)
408
409         detail = (f"{result.resolution_label or '?'}"
410                 f"{f' {result.fps:g}fps' if result.fps else ''},
{result.size_mb:.0f} MB")
411         results.append((video.video_id, "fetched", detail))
412         if args.sleep:
413             time.sleep(args.sleep)
414
415         print("\n=== Fetch summary ===")
416         print(tabulate([[vid, action, detail] for vid, action, detail in results],
417                       headers=["Video ID", "Action", "Detail"], tablefmt="grid"))
418         fetched = sum(1 for _, a, _ in results if a == "fetched")
419         failed = [r for r in results if r[1] == "FAIL"]
420         print(f"\nFetched/updated : {fetched}    Already-good : {sum(1 for _, a, _ in
results if a=='have')}}    "
421             f"Skipped : {sum(1 for _, a, _ in results if a=='skip')}}    Failed :
{len(failed)}}")
422         if failed:
423             print("Failures (left unchanged in DB):")
424             for vid, _, detail in failed:
425                 print(f"    - {vid}: {detail}")
426         print()
427
428     def export_eval_set(self, args):
429         """Export curated annotations as indexer-ready eval/training sets.
430
431         Emits, per qualifying video, a `

```

```

465         for video in videos:
466             if commercial_only and not video.is_commercial:
467                 skipped_noncomm += 1
468                 continue
469
470             intervals = self.db.get_segment_intervals(video.video_id, sport)
471             if not intervals:
472                 skipped_empty += 1
473                 continue
474
475             sport_dir = os.path.join(out_dir, sport.value)
476             os.makedirs(sport_dir, exist_ok=True)
477             csv_path = os.path.join(sport_dir, f"{video.video_id}.csv")
478             with open(csv_path, "w", newline="", encoding="utf-8") as f:
479                 writer = csv.writer(f)
480                 writer.writerow(["start", "end"])
481                 for start, end in intervals:
482                     writer.writerow([start, end])
483
484             manifest.append({
485                 "video_id": video.video_id,
486                 "sport": sport.value,
487                 "csv": os.path.relpath(csv_path, out_dir).replace(os.sep, "/"),
488                 "local_path": video.local_path,
489                 "video_url": video.video_url,
490                 "match_name": video.match_name,
491                 "fps": video.fps,
492                 "resolution": video.resolution,
493                 "license_type": video.license_type.value,
494                 "is_commercial": video.is_commercial,
495                 "status": video.status.value,
496                 "num_segments": len(intervals),
497             })
498
499         os.makedirs(out_dir, exist_ok=True)
500         manifest_doc = {
501             "commercial_only": commercial_only,
502             "statuses": [s.value for s in statuses],
503             "total_videos": len(manifest),
504             "total_segments": sum(m["num_segments"] for m in manifest),
505             "eval_sets": manifest,
506         }
507         manifest_path = os.path.join(out_dir, "manifest.json")
508         with open(manifest_path, "w", encoding="utf-8") as f:
509             json.dump(manifest_doc, f, indent=2)
510
511         print(f"\n=== Exported eval/training set to '{out_dir}' ===")
512         if manifest:
513             summary = [[m["sport"], m["video_id"], m["num_segments"],
514                         "YES" if m["is_commercial"] else "NO",
515                         os.path.basename(m["local_path"]) if m["local_path"] else "(no
local file)"]
516                        for m in manifest]
517             print(tabulate(summary, headers=["Sport", "Video ID", "Segments", "Comm?",
"Video file"], tablefmt="grid"))
518             print(f"\nVideos exported : {len(manifest)}")
519             print(f"Total segments : {manifest_doc['total_segments']}")
520             if skipped_noncomm:
521                 print(f"Skipped (non-commercial) : {skipped_noncomm} (pass --include-
noncommercial to include)")
522             if skipped_empty:
523                 print(f"Skipped (no annotations) : {skipped_empty}")
524             print(f"Manifest : {manifest_path}\n")
525
526     def convert_cvat(self, args):

```

```

527         """Convert a vendor CVAT rally export into our canonical per-rally CSV.
528
529         Absorbs the CVAT bounding-box delivery format (rally fields carried as box
530         attributes) on the ingestion side: each rally's time is recomputed from
531         its frame range at the video's true fps, and content defects (overlaps,
532         zero-duration, off-vocabulary endings, coarse sampling) are written to an
533         issues report rather than silently "fixed". The emitted CSV imports via
534         `import-local`; any blocker issue must be resolved first, since the
535         importer's validator rejects overlapping / zero-duration rallies.
536         """
537         from src.parsers.cvat.rally_cvat import (
538             convert, write_canonical_csv, render_issue_report,
539         )
540         try:
541             result = convert(args.input, fps=args.fps,
542                             video_path=args.video_path, step=args.step)
543         except (FileNotFoundError, ValueError) as e:
544             print(f"Error: {e}")
545             sys.exit(1)
546
547         write_canonical_csv(result.records, args.out)
548         report = render_issue_report(result, os.path.basename(args.input))
549         if args.issues:
550             with open(args.issues, "w", encoding="utf-8") as f:
551                 f.write(report)
552         print(report)
553         print(f"Canonical CSV written: {args.out} ({len(result.records)} rallies)")
554         if result.n_blockers:
555             print(f"\n {result.n_blockers} BLOCKER issue(s) ? import-local will reject
556 "
557                 f"this file until they are resolved (by the vendor or manual
558 review).")
559         else:
560             print("\n No blockers. Next:")
561             print(f"    python -m src.cli.curate import-local --sport <sport> "
562                   f"--video-path <video> --annotations-path {args.out} --fps
563 {result.fps:g}")
564
565     def validate_delivery(self, args):
566         """Run the full validation suite on a rally delivery and emit a report.
567
568         Accepts a CVAT export (--input + --video-path/--fps) or an already-canonical
569         CSV (--csv). Writes, into --out-dir:
570         <id>_report.md    human report (scoreboard + issues + rallies to review)
571         <id>_report.json  machine summary (for CI / a bulk-ingest loop)
572         <id>_review.csv   the annotatable manual-verification sheet
573         Exits non-zero if any blocker issue exists, so it can gate a bulk pipeline.
574         """
575         import json
576         from src.parsers.cvat.rally_cvat import (
577             convert, load_canonical_csv, detect_issues,
578             build_summary, render_validation_md, write_review_csv,
579         )
580         vid = args.video_id or "delivery"
581         fps = None
582         if args.input:
583             try:
584                 result = convert(args.input, fps=args.fps, video_path=args.video_path)
585             except (FileNotFoundError, ValueError) as e:
586                 print(f"Error: {e}")
587                 sys.exit(1)
588             records, issues, fps = result.records, result.issues, result.fps
589         elif args.csv:
590             try:
591                 records = load_canonical_csv(args.csv)

```

```

589         except FileNotFoundError as e:
590             print(f"Error: {e}")
591             sys.exit(1)
592     if not records:
593         print(f"Error: no usable rally rows in {args.csv}")
594         sys.exit(1)
595     issues = detect_issues(records)
596 else:
597     print("Error: provide --input <cvat.xml> or --csv <canonical.csv>")
598     sys.exit(1)
599
600     os.makedirs(args.out_dir, exist_ok=True)
601     summary = build_summary(records, issues, source=vid, fps=fps)
602     md = render_validation_md(records, issues, source=vid)
603     json_path = os.path.join(args.out_dir, f"{vid}_report.json")
604     md_path = os.path.join(args.out_dir, f"{vid}_report.md")
605     review_path = os.path.join(args.out_dir, f"{vid}_review.csv")
606     with open(json_path, "w", encoding="utf-8") as f:
607         json.dump(summary, f, indent=2)
608     with open(md_path, "w", encoding="utf-8") as f:
609         f.write(md)
610     write_review_csv(records, issues, review_path)
611
612     print(md)
613     print(f"\nwrote:\n {md_path}\n {json_path}\n {review_path}")
614     if summary["n_blockers"]:
615         print(f"\n GATE: BLOCKED ? {summary['n_blockers']} blocker(s) must be
resolved.")
616         sys.exit(2)
617     print("\n GATE: PASS ? review the flagged rallies, then `import-local`.")
618
619 def curate(self):
620     """Interactive loop to review and curate staging video annotations."""
621     staging_videos = self.db.get_videos_by_status(CurationStatus.STAGING)
622     if not staging_videos:
623         print("\nNo staging video annotations to review.\n")
624         return
625
626     print(f"\nFound {len(staging_videos)} matches in STAGING state for curation:")
627     table_data = []
628     for idx, video in enumerate(staging_videos):
629         table_data.append([
630             idx + 1,
631             video.video_id,
632             video.match_name,
633             video.sport.value,
634             video.license_type.value,
635             "YES" if video.is_commercial else "NO"
636         ])
637
638     print(tabulate(table_data, headers=["#", "Video ID", "Match Name", "Sport",
"License", "Commercial Safe?"], tablefmt="grid"))
639     print("\nEnter a number (1-{}) to review details, or 'q' to
quit:".format(len(staging_videos)))
640
641     choice = input("> ").strip()
642     if choice.lower() == 'q':
643         return
644
645     try:
646         idx = int(choice) - 1
647         if idx < 0 or idx >= len(staging_videos):
648             print("Invalid choice.")
649             return
650         self._curate_video_interactive(staging_videos[idx])

```

```

651         except ValueError:
652             print("Invalid input.")
653
654     def _curate_video_interactive(self, video: VideoMetadata):
655         """Displays details of a specific staging video and prompts for curation
656         action."""
657
658         # Load and count events
659         rallies = []
660         pts = []
661         delivs = []
662         if video.sport == SportType.BADMINTON:
663             rallies = self.db.get_badminton_rallies(video.video_id)
664         elif video.sport == SportType.PICKLEBALL:
665             rallies = self.db.get_pickleball_rallies(video.video_id)
666         elif video.sport == SportType.TABLE_TENNIS:
667             rallies = self.db.get_tabletennis_rallies(video.video_id)
668         elif video.sport == SportType.TENNIS:
669             pts = self.db.get_tennis_points(video.video_id)
670         elif video.sport == SportType.CRICKET:
671             delivs = self.db.get_cricket_deliveries(video.video_id)
672
673         while True:
674             print("\n===== REVIEWING VIDEO =====")
675             print(f"Match Name : {video.match_name}")
676             print(f"Video ID : {video.video_id}")
677             print(f"Sport : {video.sport.value}")
678             print(f"Video URL : {video.video_url or 'N/A'}")
679             print(f"Local Path : {video.local_path or 'N/A'}")
680             print(f"License : {video.license_type.value} ({video.license_url or 'no
681             URL'})")
682             print(f"Commercial OK: {video.is_commercial}")
683
684             if video.sport in (SportType.BADMINTON, SportType.PICKLEBALL,
685             SportType.TABLE_TENNIS):
686                 print(f"Total Rallies: {len(rallies)}")
687                 if rallies:
688                     print("Sample rallies:")
689                     sample_data = [[r.rally_number, f"{r.start_time}s - {r.end_time}s",
690                     r.score_after, r.shots_count] for r in rallies[:5]]
691                     print(tabulate(sample_data, headers=["Rally #", "Duration", "Score",
692                     "Shots"], tablefmt="simple"))
693                 elif video.sport == SportType.TENNIS:
694                     print(f"Total Points : {len(pts)}")
695                     if pts:
696                         print("Sample points:")
697                         sample_data = [[p.point_number, f"{p.start_time}s - {p.end_time}s",
698                         p.game_score] for p in pts[:5]]
699                         print(tabulate(sample_data, headers=["Point #", "Duration",
700                         "Score"], tablefmt="simple"))
701                 elif video.sport == SportType.CRICKET:
702                     print(f"Total Balls : {len(delivs)}")
703                     if delivs:
704                         print("Sample deliveries:")
705                         sample_data = [[f"{d.over}.{d.ball}", f"{d.start_time}s -
706                         {d.end_time}s", d.runs, "Wicket" if d.wicket else ""] for d in delivs[:5]]
707                         print(tabulate(sample_data, headers=["Over.Ball", "Duration",
708                         "Runs", "Wicket"], tablefmt="simple"))
709
710             print("\nChoose Curation Action:")
711             print(" [a] Approve (promote to CURATED)")
712             print(" [r] Reject (delete from database)")
713             print(" [p] Play a rally/point segment in browser")
714             print(" [k] Keep in Staging (do nothing and go back)")

```

```

707         action = input("> ").strip().lower()
708         if action == 'a':
709             self.db.update_status(video.video_id, CurationStatus.CURATED)
710             print(f"Successfully promoted '{video.match_name}' to CURATED.\n")
711             break
712         elif action == 'r':
713             self.db.delete_video(video.video_id)
714             print(f"Successfully rejected and deleted '{video.match_name}'.\n")
715             break
716         elif action == 'k':
717             print("Curation state unchanged.\n")
718             break
719         elif action == 'p':
720             self._play_segment_interactive(video, rallies, pts, delivs)
721         else:
722             print("Invalid action selected.")
723
724     def _play_segment_interactive(self, video: VideoMetadata, rallies:
List[BadmintonRally], pts: List[TennisPoint], delivs: List[CricketDelivery]):
725         import webbrowser
726
727         # Get start/end based on user input
728         start_time = None
729         end_time = None
730         label = "segment"
731
732         if video.sport in (SportType.BADMINTON, SportType.PICKLEBALL,
SportType.TABLE_TENNIS):
733             if not rallies:
734                 print("No rallies to play.")
735                 return
736             idx_str = input(f"Enter Rally number to play (1-{len(rallies)}): ").strip()
737             try:
738                 rally_num = int(idx_str)
739                 rally = next((r for r in rallies if r.rally_number == rally_num), None)
740                 if rally:
741                     start_time = rally.start_time
742                     end_time = rally.end_time
743                     label = f"Rally {rally_num}"
744             except ValueError:
745                 pass
746         elif video.sport == SportType.TENNIS:
747             if not pts:
748                 print("No points to play.")
749                 return
750             idx_str = input(f"Enter Point number to play (1-{len(pts)}): ").strip()
751             try:
752                 pt_num = int(idx_str)
753                 pt = next((p for p in pts if p.point_number == pt_num), None)
754                 if pt:
755                     start_time = pt.start_time
756                     end_time = pt.end_time
757                     label = f"Point {pt_num}"
758             except ValueError:
759                 pass
760         elif video.sport == SportType.CRICKET:
761             if not delivs:
762                 print("No deliveries to play.")
763                 return
764             over_str = input("Enter Over number: ").strip()
765             ball_str = input("Enter Ball number: ").strip()
766             try:
767                 o_num = int(over_str)
768                 b_num = int(ball_str)
769                 d = next((x for x in delivs if x.over == o_num and x.ball == b_num),

```

```

None)
770         if d:
771             start_time = d.start_time
772             end_time = d.end_time
773             label = f"Over {o_num} Ball {b_num}"
774     except ValueError:
775         pass
776
777     if start_time is None or end_time is None:
778         print("Segment not found or invalid input.")
779         return
780
781     print(f"Playing {label} (start: {start_time}s, end: {end_time}s)...")
782
783     # Check video location
784     play_url = None
785     if video.local_path and os.path.exists(video.local_path):
786         abs_path = os.path.abspath(video.local_path)
787         # Create file:// URL with media fragment (browsers support #t=start,end
fragment)
788         play_url = f"file:/// {abs_path.replace(os.sep,
'/' )}#t={start_time},{end_time}"
789     elif video.video_url:
790         # YouTube URL: seek to start time
791         if "youtube.com" in video.video_url or "youtu.be" in video.video_url:
792             t_sec = int(start_time)
793             sep = "&" if "?" in video.video_url else "?"
794             play_url = f"{video.video_url}{sep}t={t_sec}s"
795         else:
796             play_url = video.video_url
797
798     if play_url:
799         print(f"Opening browser: {play_url}")
800         webbrowser.open(play_url)
801     else:
802         print("Error: No local video file or remote URL available to play.")
803
804
805     def main():
806         parser = argparse.ArgumentParser(description="Sports Data Curator CLI")
807         subparsers = parser.add_subparsers(dest="command")
808
809         # Status command
810         subparsers.add_parser("status", help="Display summary status of sports metadata
database")
811
812         # Curate command
813         subparsers.add_parser("curate", help="Launch interactive curation tool for staging
entries")
814
815         # Import-local command
816         import_parser = subparsers.add_parser("import-local", help="Manually import a local
video and annotations")
817         import_parser.add_argument("--sport", required=True, help="Sport type (e.g.
badminton, tennis, cricket)")
818         import_parser.add_argument("--video-path", required=True, help="Path to local video
file")
819         import_parser.add_argument("--annotations-path", required=True, help="Path to local
JSON/CSV annotations file")
820         import_parser.add_argument("--match-name", help="Name of the match (defaults to
video filename)")
821         import_parser.add_argument("--video-id", help="Custom alphanumeric video ID")
822         import_parser.add_argument("--license", default="Proprietary", help="License type
(e.g. MIT, CC-BY-NC, Proprietary)")
823         import_parser.add_argument("--fps", type=float, default=30.0, help="Frames per

```

```

second of the video")
824     import_parser.add_argument("--resolution", default="1080p", help="Resolution of the
video (e.g., 1080p, 720p)")
825
826     # Fetch command ? (re)download videos at best resolution, update DB in place
827     fetch_parser = subparsers.add_parser(
828         "fetch", help="(Re)download videos at best available resolution, updating the DB
in place")
829     fetch_parser.add_argument("--sport", help="Restrict to a single sport (default: all
supported_sports)")
830     fetch_parser.add_argument("--video-id", help="Fetch a single video by ID")
831     fetch_parser.add_argument("--status", choices=["staging", "curated", "all"],
default="all",
832                                     help="Which curation statuses to fetch (default: all)")
833     fetch_parser.add_argument("--min-height", type=int, default=720,
834                               help="Skip videos already at/above this height unless
--force (default: 720)")
835     fetch_parser.add_argument("--max-height", type=int, default=None,
836                               help="Cap download resolution (e.g. 1080); default: best
available")
837     fetch_parser.add_argument("--force", action="store_true",
838                               help="Re-download even if a good-enough local file already
exists")
839     fetch_parser.add_argument("--include-noncommercial", action="store_true",
840                               help="Include videos whose license is not commercially
safe (default: commercial only)")
841     fetch_parser.add_argument("--cookies-from-browser", dest="cookies_from_browser",
metavar="BROWSER",
842                               help="Use a logged-in browser's YouTube cookies (e.g.
chrome, edge, firefox, brave). "
843                                   "A Premium login gives reliable best-resolution
downloads. "
844                                   "Chromium browsers must be CLOSED on Windows (cookie
DB lock).")
845     fetch_parser.add_argument("--cookies", metavar="FILE",
846                               help="Path to a Netscape-format cookies.txt (alternative
to --cookies-from-browser)")
847     fetch_parser.add_argument("--player-client", dest="player_client",
default="default", metavar="CLIENT",
848                               help="YouTube player client yt-dlp uses (default:
'default'). Avoid web/web_safari "
849                                   "(SABR-forced -> drops to 144p). 'tv'/'mweb' also
serve 1080p if needed.")
850     fetch_parser.add_argument("--sleep", type=float, default=4.0, metavar="SECONDS",
851                               help="Pause between videos to avoid YouTube rate-limiting
(default: 4)")
852     fetch_parser.add_argument("--retries", type=int, default=2, metavar="N",
853                               help="Retries per video when a download fails or returns a
low format (default: 2)")
854     fetch_parser.add_argument("--retry-backoff", type=float, default=20.0,
metavar="SECONDS",
855                               help="Base backoff between retries; grows linearly per
attempt (default: 20)")
856
857     # Convert-cvat command ? normalize a vendor CVAT export into our canonical CSV
858     cvat_parser = subparsers.add_parser(
859         "convert-cvat",
860         help="Convert a vendor CVAT rally export (bounding-box XML) into the canonical
per-rally CSV")
861     cvat_parser.add_argument("--input", required=True,
862                               help="Path to the CVAT XML export (image- or track-mode)")
863     cvat_parser.add_argument("--out", required=True,
864                               help="Path to write the canonical per-rally CSV (import-
local input)")
865     cvat_parser.add_argument("--video-path",

```

```

866             help="Source video; its fps is read via ffprobe (seconds =
frame / fps)")
867         cvat_parser.add_argument("--fps", type=float,
868             help="Override fps instead of probing the video")
869         cvat_parser.add_argument("--step", type=int,
870             help="Frame sampling step (default: read from CVAT meta,
else inferred)")
871         cvat_parser.add_argument("--issues",
872             help="Path to also write the human-readable issues report")
873
874         # Validate-delivery command ? run the full check suite + emit a structured report
875         vd_parser = subparsers.add_parser(
876             "validate-delivery",
877             help="Validate a rally delivery (CVAT export or canonical CSV) and emit report +
review sheet")
878         vd_parser.add_argument("--input", help="CVAT XML export (use with --video-
path/--fps)")
879         vd_parser.add_argument("--csv", help="An already-canonical per-rally CSV to
validate")
880         vd_parser.add_argument("--video-path",
881             help="Source video; fps via ffprobe (when using --input)")
882         vd_parser.add_argument("--fps", type=float, help="Override fps (when using
--input)")
883         vd_parser.add_argument("--video-id", help="Names the report files (<id>_report.md,
etc.)")
884         vd_parser.add_argument("--out-dir", default="data/validation",
885             help="Where to write the report + review sheet")
886
887         # Export command
888         export_parser = subparsers.add_parser(
889             "export", help="Export curated annotations as indexer-ready eval/training CSVs +
manifest")
890         export_parser.add_argument("--out-dir", default="data/eval_export",
891             help="Output directory for per-video CSVs and
manifest.json")
892         export_parser.add_argument("--sport",
893             help="Restrict to a single sport (default: all
supported_sports)")
894         export_parser.add_argument("--include-staging", action="store_true",
895             help="Also export videos still in staging (default:
curated only)")
896         export_parser.add_argument("--include-noncommercial", action="store_true",
897             help="Include videos whose license is not commercially
safe (default: commercial only)")
898
899         args = parser.parse_args()
900         cli = SportsDataCLI()
901
902         if args.command == "status":
903             cli.status()
904         elif args.command == "curate":
905             cli.curate()
906         elif args.command == "import-local":
907             cli.import_local(args)
908         elif args.command == "convert-cvat":
909             cli.convert_cvat(args)
910         elif args.command == "validate-delivery":
911             cli.validate_delivery(args)
912         elif args.command == "export":
913             cli.export_eval_set(args)
914         elif args.command == "fetch":
915             cli.fetch(args)
916         else:
917             # Default: print help
918             parser.print_help()

```

```

919
920     if __name__ == "__main__":
921         main()
922

```

4. user (2026-06-06T17:32:38.429Z)

```

1      # Illustrated Annotation Guide ? Rally Labeling
2
3      Supplement to [ `R00RA_BRIEF.md` ] (./R00RA_BRIEF.md). Sports: **Badminton, Pickleball,
Table Tennis** (singles **and** doubles).
4
5      > **Illustrations:** diagrams are drawn in text so they survive copy/paste. Boxes marked
6      > `[SCREENSHOT: ...]` are placeholders.
7      > **TODO(avidullu):** replace every `[SCREENSHOT: ...]` placeholder with a real frame grab
8      > from the pilot videos before sending this guide to Roora.**
9
10     ## Part 1 ? What you are producing
11     One CSV per video (from the template Sheet), one row per rally:
12
13     ```
14     rally_number, start_mmss, end_mmss, start_time, end_time, shots_count, ending_reason,
last_shot_outcome, score_before, score_after, notes
15     ```
16
17     You type: `rally_number`, `start_mmss`, `end_mmss`, `shots_count`, `ending_reason`,
18     `last_shot_outcome` (+ optional score/notes). The `start_time`/`end_time` decimal-second
columns
19     **fill themselves** via a formula already in the template.
20
21     For every rally you record:
22     - **When it started** (serve contact) ? `start_mmss` (`m:ss.mmm`, e.g. `1:12.40`)
23     - **When it ended** (shuttle/ball dead) ? `end_mmss`
24     - **How many shots** happened ? `shots_count`
25     - **Why it ended** ? `ending_reason` . **Where the last shot landed** ?
`last_shot_outcome`
26
27     Everything else (gaps between rallies) you **ignore**.
28
29     ## Part 2 ? The three golden rules
30
31     ### Rule 1: Ignore all dead time
32     A rally is **continuous active play**. The moment play stops, the rally is over. The
time
33     until the next serve is **dead time** and is never labeled.
34
35     ```
36     DEAD          RALLY 1          DEAD          RALLY 2          DEAD
37     (ignore) |=====|          (ignore) |=====|          (ignore)
38             ^                   ^                   ^                   ^
39             serve                shuttle              serve                shuttle
40             contact               lands                 contact               lands
41             =start_1              =end_1                 =start_2              =end_2
42     ```
43
44     Dead time includes: walking back to serve, picking up the shuttle/ball, wiping sweat,
45     bouncing the ball before serving, the score being announced, replays/slow-mo, crowd or
coach
46     shots, changeovers, towel/medical timeouts, and **pre-match / between-games warm-up
47     (knock-up)** ? see the Rule 2 caveat below.
48
49     ### Rule 2: Include every rally ? even the ugly ones
50     Every served point gets a row, including:
51     - **Service faults** (serve into net/out/illegal) ? usually `shots_count = 1`,
`ending_reason = service_fault`.

```

```

52 - **Very short rallies** (serve + one error).
53 - **Lets / replays** ? `ending_reason = let`; the replayed point is its **own** next
row.
54
55 A **missed rally is the worst error** in this project ? the model must learn that short
and
56 failed rallies are still rallies.
57
58 > **?? BUT warm-up / knock-up is NOT a rally** ? a common trap, because Rule 2 says
"include
59 > everything." Pre-match and between-games warm-up *looks* like rallying but is not
scored play;
60 > **do not label it.** How to tell it apart:
61 > - **No real serve.** Real play starts with a proper serve from the service position
(diagonal,
62 > behind the service line). Warm-up has players feeding cooperatively from mid-court
with no
63 > serve ? not a rally.
64 > - **Cooperative, not competitive.** Gentle hits straight to each other; nobody is
trying to win
65 > the point (no smashes-to-win, no scrambling/diving).
66 > - **Score isn't progressing.** No umpire / scoreboard not running / players casually
retrieve
67 > and re-feed; the score doesn't change. Often **several shuttles/balls** in use at
once.
68 > - **Per sport:** badminton & pickleball ? real play starts with the underhand serve
from the
69 > service court / behind the baseline; table tennis ? real play starts with the
tossed,
70 > two-bounce serve (knock-up is cooperative rallying without it).
71 >
72 > **When unsure:** find the first proper serve where the score starts to progress ?
labeling
73 > begins there. If you genuinely can't tell whether the match has started, **flag the
clip to us
74 > ? don't guess.**
75
76 ### Rule 3: A shot = one contact with the racket / paddle / bat
77 Count every time the shuttle/ball touches a racket, paddle, or bat. The serve is shot 1.
78 - **Count:** any racket/paddle/bat contact (serve, return, smash, block?).
79 - **Don't count:** floor bounces, table bounces, net clips, the toss before a serve.
80
81 5-shot example: `serve(1) ? return(2) ? hit(3) ? hit(4) ? winner(5)` ? `shots_count =
5`, `ending_reason = winner`.
82
83 ## Part 3 ? Where exactly does a rally start and end?
84 **START (`start_mmss`):** the exact frame the server's racket/paddle/bat **contacts**
the
85 shuttle/ball. Not the toss, not the wind-up ? the contact frame.
86 `[SCREENSHOT: serve-contact frame]`
87
88 **END (`end_mmss`):** the exact frame the shuttle/ball becomes **dead**:
89 - Badminton: shuttle touches the floor, or play stops on a fault.
90 - Pickleball: the second floor bounce, or the frame the ball hits the net / lands out.
91 - Table tennis: the second bounce on one side, a non-serve net hit, or off the table.
92
93 `[SCREENSHOT: end frame]`
94
95 If occluded/off-screen, pick the closest clear frame and add a note.
96 **Boundary tolerance: ±0.3 s** (~±9 frames at 30 fps).
97
98 ## Part 4 ? ending_reason: how to choose
99 Pick exactly **one** value for how the rally ended. Stop at the first match:
100
101 1. Did the **serve** itself fail / was illegal (and ended the point)? ? `service_fault`

```

```

102     2. Was the point **replayed** (let / interruption)? ? `let`
103     3. Did the final shot land **in** and the opponent make **no legal contact**? ? `winner`
(a clean winner)
104     4. Did the losing player make an error (netted / out / missed) **under pressure** ?
105         stretched, wrong-footed, or rushed by a strong shot? ? `forced_error`
106     5. Did the losing player make an error on a **routine, unpressured** ball they had
107         the time and position to make? ? `unforced_error`
108     6. None of the above / cannot tell ? `other` (add a note)
109
110     **Forced vs unforced ? the one judgment call.** Decide using the opponent's *preceding*
111     shot:
112     - `forced_error` = the opponent's shot **caused** the miss (pace, placement, deception;
113         the player barely reached it or was pulled out of position).
114     - `unforced_error` = a clean, makeable ball the player missed on their own.
115     - When genuinely unsure between the two, choose `unforced_error` and add a brief note.
116
117     Notes:
118     - A clean `winner` means **no legal contact** by the opponent. If they got a racket /
paddle /
119         bat on it but netted or hit out, that is a `forced_error` / `unforced_error` (their
shot), not a `winner`.
120     - A serve into the net is `service_fault` (rule 1 beats everything).
121
122     ### Then set `last_shot_outcome` ? where the last shot physically went
123     This is a **separate, observational** column (no judgment): just look at the last shot.
124     - `in` ? landed in (winners; also a put-away the opponent couldn't reach).
125     - `net` ? went into the net.
126     - `out` ? landed outside the court / past the lines.
127     - `n_a` ? not applicable: use for `service_fault` and `let`.
128
129     So each rally gets both: **why** (`ending_reason`) and **where** (`last_shot_outcome`) ?
e.g. a
130     smash winner = `winner, in`; a clear forced long = `forced_error, out`; a routine ball
netted =
131     `unforced_error, net`.
132
133     ## Part 5 ? Per-sport detail
134
135     ### Badminton
136     - **Serve:** underhand, below the waist, diagonal. Start = racket?shuttle contact.
137     - **Rally end:** shuttle hits the floor, or play stops on a fault.
138     - **Shots:** every racket?shuttle contact; serve = shot 1.
139     - **Service faults** (1-shot rally, `service_fault`): serve into net, serve outside
service court, contact above waist, foot fault.
140
141     Worked example:
142     ```
143     [SCREENSHOT: badminton serve]
144     Rally 4: serve 1:28.30, long rally, smash winner, shuttle lands 1:44.10, 12 contacts.
145         4, 1:28.30, 1:44.10, (auto), (auto), 12, winner, 3-5, 4-5,
146     Rally 5: serve 1:57.00 straight into the net.
147         5, 1:57.00, 1:57.90, (auto), (auto), 1, service_fault, 4-5, 4-6,
148     ```
149
150     **Badminton ? DOUBLES (2 players per side).** All rally rules are **identical** to
singles:
151     - `start` = serve contact; `end` = shuttle down / fault; a **shot = any racket?shuttle
contact by either player on a side**.
152     - `shots_count` = **total** contacts across both teams in the rally. **Do not track
which player hit** ? just count every contact.
153     - Net exchanges (drives, flat pushes) come fast ? **frame-step** so you neither miss nor
double-count contacts.
154     - Only the **designated receiver** returns the serve; if the wrong player returns, the
rally ends on a fault ? still a row (`service_fault` if it's a serve/receiving-position fault,
else `other` + note).

```

```

155     - Two teammates may **not** both hit the shuttle in succession (double-hit). If you see
two contacts on the **same** side back-to-back, that's a fault ending the rally ? count both
contacts, then the rally ends.
156
157     ### Pickleball
158     - **Serve:** underhand, paddle below waist, hit diagonally; must clear the non-volley
zone (the "kitchen"). Start = paddle?ball contact.
159     - **Two-bounce rule:** the serve must bounce once in the receiver's court and the return
must bounce too, before either side may volley. These bounces are **normal play**, not rally
ends, and bounces are **not** shots.
160     - **Rally end:** ball bounces twice, lands out, hits the net (fails to cross), or a
fault (e.g. volleying in the kitchen).
161     - **Shots:** every paddle?ball contact only; serve = shot 1; floor bounces are not
shots.
162     - **Service faults** (`service_fault`): serve into net, out/long, short in the kitchen,
foot fault.
163
164     Worked example:
165     ```
166     [SCREENSHOT: pickleball serve + kitchen line]
167     Rally 2: serve 0:33.10; serve bounces, return bounces, a few volleys; receiver nets an
easy one at 0:41.85; 6 paddle contacts.
168     2, 0:33.10, 0:41.85, (auto), (auto), 6, unforced_error, 1-0, 2-0, easy ball netted
169     ```
170
171     **Pickleball ? DOUBLES (the standard format, 2 per side).** All rally rules
**identical** to singles:
172     - `start` = serve contact; a **shot = any paddle?ball contact by _either_ teammate**;
floor bounces still not shots; two-bounce rule still applies.
173     - `shots_count` = **total** paddle contacts across both teams; don't track which player.
174     - **Serving sequence (FYI only ? you do NOT annotate it):** both partners serve before
the serve passes to the opponents, except the very first service turn of the game. The called
score has three numbers (server score ? receiver score ? server #1/#2). Ignore this for
annotation; just mark boundaries / shots / ending.
175     - If both teammates contact the ball in succession, that's a fault ? rally ends; count
the contacts, set the ending per the fault.
176
177     ### Table tennis
178     - **Serve:** ball tossed up from an open palm, then struck so it bounces once on each
side. Start = the **racket?ball strike** (not the toss).
179     - **Rally end:** a player fails a legal return ? ball double-bounces on their side, goes
off the table, or is hit into the net on a non-serve shot.
180     - **Shots:** every racket?ball contact only; serve = shot 1; table bounces are not
shots.
181     - **Let:** a serve that clips the net but is otherwise legal ? the point is replayed.
`ending_reason = let`, and the replay is its own next row.
182     - **Service faults** (`service_fault`): missed/illegal toss, serve into net, serve off
the table.
183     - **Speed warning:** contacts can be < 0.2 s apart ? **step frame-by-frame** to count
shots.
184     - **Doubles (if a clip appears):** partners must hit in **strict alternation** and serve
diagonally; a missed alternation is a fault that ends the rally. Same rule ? count **every**
racket?ball contact as a shot; `shots_count` is the team total.
185
186     Worked example:
187     ```
188     [SCREENSHOT: table-tennis serve contact]
189     Rally 9: serve 4:10.40; fast 7-contact rally; forehand winner, opponent no contact; dead
4:14.05.
190     9, 4:10.40, 4:14.05, (auto), (auto), 7, winner, 6-9, 6-10,
191     Rally 10: serve 4:22.10 clips the net (otherwise good) -> let.
192     10, 4:22.10, 4:23.00, (auto), (auto), 1, let, 6-10, 6-10, net-cord serve, replayed
193     ```
194
195     ## Part 6 ? Edge cases & FAQ

```

```

196     - **Two rallies with almost no gap?** Still two rows. `end_time[N] <= start_time[N+1]`,
never overlap.
197     - **Interrupted & replayed?** Label the interrupted attempt as its own row
(`ending_reason=let`), then the replayed point as the next row.
198     - **Serve tossed but caught / re-served (no contact)?** No contact = no rally yet. Start
at the actual serve contact; don't label the aborted toss.
199     - **Opponent reached the ball but netted/hit out?** That's a
`forced_error`/`unforced_error` (their shot), not `winner` ? `forced` if your shot pressured
them, `unforced` if it was routine.
200     - **Can't read the scoreboard?** Leave `score_before`/`score_after` blank. Never guess.
201     - **"Let" where the rules say play on?** If play continued, it's **not** a let ? keep
one rally and judge the real ending.
202     - **Warm-up / knock-up / practice points?** Not real rallies ? **do not label them.**
Start at the first real, scored serve. See the **Rule 2 caveat (Part 2)** for how to tell warm-
up from real play (no serve, cooperative not competitive, score not progressing, several
shuttles/balls in use). When unsure whether the match has started, flag the clip ? don't guess.
203     - **Doubles?** Same rules ? a shot is any one racket/paddle contact by whichever player
hits it. See the badminton and pickleball **DOUBLES** notes in Part 5; never attribute
`shots_count` to an individual, just total the contacts.
204     - **Server completely whiffs (no contact)?** Rare. True zero contact ? `shots_count =
0`, `ending_reason = service_fault`, add a note. A graze = 1 contact = 1 shot.
205
206     ## Part 7 ? Self-check before you submit (per video)
207     - [ ] `rally_number` is 1,2,3? no gaps or repeats.
208     - [ ] Every `start_time < end_time` (check the auto-filled decimal columns populated).
209     - [ ] No overlaps: `end_time[N] <= start_time[N+1]`.
210     - [ ] Times entered as `m:ss.mmm`; decimal `start_time`/`end_time` auto-filled and look
right.
211     - [ ] No dead-time rows (scan the gaps ? dead time, not missing rallies).
212     - [ ] Every served point present, including service faults and lets.
213     - [ ] `shots_count` looks right (re-step any rally you rushed; doubles = team total).
214     - [ ] `ending_reason` ?
{`winner`, `forced_error`, `unforced_error`, `service_fault`, `let`, `other`} ; notes filled when
`other`.
215     - [ ] `last_shot_outcome` ? {`in`, `net`, `out`, `n_a`} (`n_a` for service_fault/let).
216     - [ ] File downloaded as `.csv`, header row present, UTF-8.
217     - [ ] Did **not** trim / re-encode / rename the source video.
218
219     ## Part 8 ? Mini glossary
220     - **Rally** ? one continuous passage of play from serve contact to the shuttle/ball
dead.
221     - **Shot** ? one contact between the shuttle/ball and a racket/paddle/bat (either
teammate in doubles).
222     - **Dead time** ? any time the shuttle/ball is not in active play. Never labeled.
223     - **Service fault** ? a failed/illegal serve that ends the point. Still a labeled rally.
224     - **Let** ? a replayed point. Labeled as its own rally with `ending_reason=let`.
225     - **Winner / forced_error / unforced_error** ? the final shot was a clean winner
(opponent made no legal contact) / the loser's error was forced by a pressuring shot / the
loser's error was on a routine, unpressured ball.
226     - **start_time / end_time** ? decimal seconds from the start of the provided file (auto-
filled from mm:ss).
227

```

5. user (2026-06-06T17:32:38.896Z)

```

1      # Roora Annotation Brief ? Rally Golden Set
2
3      **Project:** racket-sport rally segmentation golden / regression dataset
4      **Sports in scope:** Badminton, Pickleball, Table Tennis (each in **singles _and_
doubles**)
5      **Companion doc:** [ `ANNOTATION_GUIDE.md` ] (./ANNOTATION_GUIDE.md)
6
7      > This brief is the vendor-facing spec. The annotation deliverable is a per-rally CSV
that
8      > imports directly via `python -m src.cli.curate import-local`.

```

```

9
10  ## 0. What we need in one sentence
11  For each provided match video, produce one CSV file listing every rally with its
12  start time, end time, number of shots, and how it ended ? excluding all dead time.
13
14  ## 1. Deliverable format (canonical)
15  One CSV per video (UTF-8), named exactly `.csv` (we supply the `video_id`).
16  One row per rally, chronological, header row required.
17
18  | column | meaning |
19  |---|---|
20  | `rally_number` | integer, 1-based, chronological. No gaps, no repeats. |
21  | `start_mmss` | rater-entered time as `m:ss.mmm` (e.g. `1:12.40`) ? serve contact. |
22  | `end_mmss` | rater-entered time as `m:ss.mmm` ? ball/shuttle dead. |
23  | `start_time` | decimal seconds (e.g. `72.40`). Auto-filled from `start_mmss`.
24  | `end_time` | decimal seconds. Auto-filled from `end_mmss`. |
25  | `shots_count` | integer. Total racket/paddle/bat contacts (serve = shot 1). |
26  | `ending_reason` | one of: `winner` \| `forced_error` \| `unforced_error` \|
27  | `service_fault` \| `let` \| `other` (why it ended) |
28  | `last_shot_outcome` | one of: `in` \| `net` \| `out` \| `n_a` (where the last shot
29  landed; `n_a` for service_fault/let) |
30  | `score_before` | optional, `A-B`. Blank if not confidently readable. |
31  | `score_after` | optional, `A-B`. |
32  | `notes` | optional free text. Required when `ending_reason = other`. |
33
34  Times (mm:ss ? decimal seconds). Raters type only `start_mmss`/`end_mmss`. The
35  template
36  Google Sheet fills `start_time`/`end_time` with this formula (shown for `start_time`,
37  row 2):
38
39  ```
40  =IF(B2="", "", IFERROR(VALUE(LEFT(B2,FIND(":",B2)-1))*60 +
41  VALUE(MID(B2,FIND(":",B2)+1,20)), VALUE(B2)))
42  ```
43
44  On Download as CSV, the computed decimal seconds are exported. (If your tool already
45  shows decimal seconds, type those into `start_time`/`end_time` and leave the mm:ss
46  columns blank.)
47
48  Example (badminton):
49
50  ```csv
51  rally_number,start_mmss,end_mmss,start_time,end_time,shots_count,ending_reason,last_shot
52  _outcome,score_before,score_after,notes
53  1,0:12.50,0:25.00,12.50,25.00,9,winner,in,0-0,1-0,
54  2,0:45.00,0:47.30,45.00,47.30,1,service_fault,n_a,1-0,1-1,serve into net
55  3,1:01.20,1:18.85,61.20,78.85,14,unforced_error,out,1-1,2-1,final clear sailed long (no
56  pressure)
57  ```
58
59  Critical format rules
60
61  - `start_time`/`end_time` that reach us must be decimal seconds (the formula
62  guarantees this).
63
64  - Times are relative to the start of the file we provide. Do not trim, clip, or
65  re-encode the video ? our pipeline identifies a video by its file checksum.
66
67  - `end_time` > `start_time` for every row.
68
69  - No overlaps: `end_time[N]` <= `start_time[N+1]`.
70
71  - Everything between one rally's `end_time` and the next rally's `start_time` is dead
72  time and gets no row.
73
74  ## 2. The three non-negotiable rules (full detail + pictures in the Guide)
75
76  1. Ignore all dead time. Only label active play.
77
78  2. Include every rally ? exclude none. Every served point, including service
79  faults,
80
81  1?2 shot rallies, and lets/replays. A missed rally is the worst error in this

```

```

project.
62     **Exception: warm-up / knock-up is NOT a rally** ? pre-match and between-games warm-
up looks
63     like rallying but isn't scored play, so **don't label it**. Tell it apart by: no real
serve,
64     cooperative (not competitive) hitting, the score not progressing, often several
shuttles/balls
65     in use (full how-to in the Guide, Part 2). When unsure if the match has started, flag
it.
66     3. **A shot = one contact** with the racket/paddle/bat. Serve = shot 1. Bounces and net-
clips are not shots.
67
68     ## 3. Scope & phasing
69     **Phase A ? Pilot (this engagement):** 3 videos (1 badminton + 1 pickleball + 1 table
tennis),
70     each `< 3 min` with ~10?12 rallies. This is the **paid calibration round**; we review
jointly and
71     lock the Guide before scale-up.
72
73     **Phase B ? Scale (after a successful pilot):** ~12?15 videos **per sport** (~36?45
total), each
74     ~30 min. Same format, rules, and template.
75
76     > **Doubles are included** across the program. Rally rules are identical to singles; the
77     > badminton- and pickleball-specific doubles notes are in the Guide.
78
79     ## 4. Quality bar (acceptance criteria)
80     Automated validation (no overlaps, positive durations, schema-valid) runs on every file,
then a
81     ~20% human spot-check. A file is returned for rework (in scope) if it misses:
82     - **Completeness:** every rally present; zero missed rallies; zero dead-time rows.
83     - **Boundaries:** `start_time`/`end_time` within  $\pm 0.3$  s of true serve-contact /
landing.
84     - **shots_count:** exact for rallies ? 15 shots;  $\pm 1$  allowed for longer/faster rallies.
85     - **ending_reason:** one of the six allowed values, per the Guide's decision tree.
86     - **Format:** decimal seconds present, valid columns, non-overlapping, chronological.
87
88     ## 5. Per-sport quick reference (full version + examples in the Guide)
89     **Badminton (singles & doubles)** ? start: racket?shuttle serve contact . end: shuttle
floors / fault . shot: each racket?shuttle contact.
90     **Pickleball (singles & doubles)** ? start: paddle?ball serve contact . end: double
bounce / out / net / fault . shot: each paddle?ball contact (floor bounces are not shots;
two-bounce rule is normal play).
91     **Table tennis** ? start: racket?ball serve strike (not the toss) . end: failed legal
return . shot: each racket?ball contact (table bounces are not shots). Net-cord serve =
`let`.
92
93     ## 6. ending_reason values (decision tree in the Guide)
94     This vocabulary attributes why the point ended; it is shared across all
95     net-separated sports (badminton, pickleball, table tennis, tennis). Record where
96     the last shot physically landed in the separate `last_shot_outcome` column
97     (`in` \| `net` \| `out` \| `n_a`) ? e.g. a forced error that flew long is
98     `ending_reason=forced_error, last_shot_outcome=out`.
99
100    | value | meaning |
101    |---|---|
102    | `winner` | last shot landed in and the opponent made no legal contact (a clean
winner). |
103    | `forced_error` | the losing player erred (netted / out / missed) under pressure ?
stretched, wrong-footed, or rushed by the opponent's strong shot. |
104    | `unforced_error` | the losing player erred on a routine, unpressured ball they had
time and position to make. |
105    | `service_fault` | the serve itself failed / was illegal. |
106    | `let` | the point was replayed. |
107    | `other` | anything else / cannot tell. Must add a note. |

```

```

108
109     ## 7. Recommended capture method
110     Use any tool that shows current time (mm:ss.mmm or seconds, or frame + known fps) and
can
111     **step one frame** forward/back: a frame-steppable player (VLC frame-step) or a timeline
tool
112     (Label Studio / CVAT). The deliverable is the CSV in the schema above.
113     `seconds = frame_number / fps` (we provide fps per video).
114
115     ## 8. Logistics & contacts
116     - **TODO(avidullu): paste the Google Drive folder link** where source videos are shared
with Roora.
117     - Per-video `fps` + `video_id`: provided in the intake manifest ? template
[`INTAKE_MANIFEST_TEMPLATE.csv`](./INTAKE_MANIFEST_TEMPLATE.csv). **TODO(avidullu): fill the
manifest (one row per pilot video) and share its link.**
118     - **TODO(avidullu): paste the Drive folder link** where Roora uploads completed
`<video_id>.csv` files.
119     - Blank annotation template Sheet (mm:ss ? seconds formula from $1 pre-filled):
**TODO(avidullu): create it and paste the link.**
120     - **TODO(avidullu): add the escalation channel** (email / WhatsApp group) for edge-case
questions.
121     - NDA before any footage is shared ? template [`NDA_TEMPLATE.md`](./NDA_TEMPLATE.md).
**TODO(avidullu): have counsel review, fill the blanks, and have Roora sign.**
122     - Roora contact on file: `info@rooragrp.com` (HSR Layout, Bangalore).
123
124     ## 9. Pricing request (please quote)
125     1. Price **per rally** and **per minute** of footage.
126     2. **Pilot** price (3 short videos, ~30?36 rallies) as a paid calibration round.
127     3. Indicative **scale** price for ~36?45 longer videos.
128     4. Turnaround for the pilot and throughput (rallies/day) at scale.
129     5. QA process and rework policy.
130     6. Whether annotators have racket-sport familiarity (helpful, not required).
131     7. Confirmation you can deliver the CSV schema above.
132

```

6. assistant (2026-06-06T17:32:42.326Z)

I have everything I need from the three files. Let me compile the exact schema.

7. user (2026-06-06T17:33:20.064Z)

Structured output provided successfully

8. assistant (2026-06-06T17:33:29.791Z)

I've extracted the exact schema. Summary of what the VLC tool must emit:

****The 8 columns `import-local` actually reads (badminton, zero transformation):****
`rally\_number, start\_time, end\_time, score\_before, score\_after, shots\_count, ending\_reason, last\_shot\_outcome` ? header is lowercased + stripped at `curate.py:224`, then read key-by-key at `curate.py:228-238`.

****Required vs optional:****

- Required: `start\_time`, `end\_time` ? decimal **seconds**, parsed by `safe\_float\_required` (blank/missing aborts the import).
- Optional: `rally\_number` (auto-numbered from row position if missing), `shots\_count` (defaults None), `score\_before`, `score\_after`, and `ending\_reason`/`last\_shot\_outcome` (pass through verbatim ? not validated by the parser).

****`ending\_reason` vocabulary (6, closed):**** `winner | forced\_error | unforced\_error | service\_fault | let | other` (ROORA_BRIEF.md:26,100-107; ANNOTATION_GUIDE.md:214).
****`last\_shot\_outcome` (4, closed):**** `in | net | out | n\_a` (`n\_a` for service_fault/let) (ROORA_BRIEF.md:27; ANNOTATION_GUIDE.md:215).

****Key gotcha:**** the importer ignores `start\_mmss`, `end\_mmss`, and `notes` ? those are only for the human Google-Sheet workflow. The VLC tool must write decimal seconds directly into `start\_time`/`end\_time`, not mm:ss.

****One-line example row**** (minimal 8-col): `1,12.50,25.00,0-0,1-0,9,winner,in`
(full canonical template form: `1,0:12.50,0:25.00,12.50,25.00,9,winner,in,0-0,1-0,`)

Files: `C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py`,
`.../docs/annotation/ANNOTATION\_GUIDE.md`, `.../docs/annotation/ROORA\_BRIEF.md`.